# cs5800-hw_3_coding_analysis

Mingxi Zhu

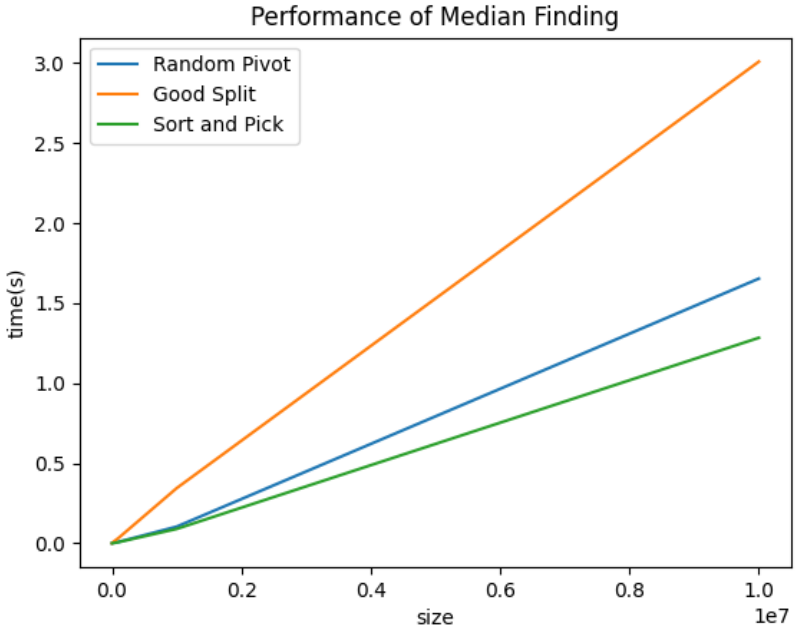
May 2026

## 1 Coding and Analysis

- **Problem 2.22 from Dasgupta et. al.**
  Part 1 and 2 is shown in problem_2_22.cpp and an executable program "problem_2_22.exe"

  Part 3:
  Because both recursion and iterating algorithms doing binary search on both arrays. They do one binary search at each array at a time. The worst case is that they need to finish the whole binary search on both arrays. Therefore, $T(n) = O(logm + logn)$.

- **Median finding**



The code work is in median_find.py

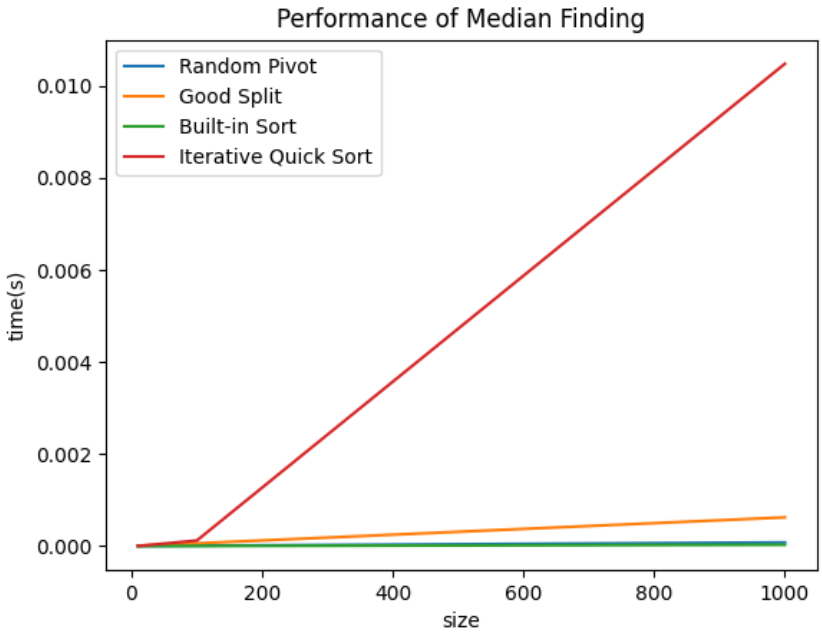
The theoretic time complexity of random pivot algorithm is $O(n \log n)$ (worst case $O(n^2)$). But the actual expected performance is $O(n)$, because the random pivot reduces the possibility of choosing a bad pivot by 50%, making it close to $T(n) = T(3n/4) + O(n)$ for the average case division.

For good split algorithm, the pivot is chosen by calculating median of a group of medians, making the case division to no more than $T(n) = T(7n/10) + T(n/5) + O(n) = T(9n/10) + O(n)$ for each iteration, converging to $O(n)$.

While the good split algorithm is guaranteed to converge to $O(n)$, the actual time cost of it is often larger than random pivot algorithm because it has a larger constant factor.

Comparing to built-in sort and pick algorithm, though the Tim Sort is between $O(n)$ and $O(n \log n)$, the cost is faster because it use C to implement.

If using python-implemented quick sort, it takes significantly longer time than all other algorithms.




# 2 Acknowledgements

### 2.0.1 Resources

median sort, Dasgupta et. al. text. Prof. Bruce's find_median algorithm BFPRT Algorithm (Median of Medians), Wikipedia

### 2.0.2 LLM Disclosure

find why the performance of good split is worse than random pivot.

# 3 Reflection

Though in theory, big-O complexity analysis can provide a good rough estimation, it is still good to know what is the best practice in real situations.